ORBIS – Smart Mini City

Project Documentation

Course	EEE 418: Laboratory on Control System
Supervisor	Saumitra Kumar Kuri, Assistant Professor
Name	Nabiha Nowrin
ID	23702051
Email	nabihanowrin0@gmail.com
Department	Electrical and Electronic Engineering
University	University of Chittagong
Team	Team ORBIS · June 2026

Contents

1	Introduction	2
2	System Architecture	4
3	Gas and Fire Monitor	6
4	RFID Door Lock	10
5	Smart Parking	15
6	Smart Dustbin	20
7	Traffic Lights	25
8	Solar Tracker	29
9	Weather Station and Central Hub	35
10	Cloud Relay, Database and Dashboard	45
11	References	48

1 Introduction

ORBIS is a working scale model of a smart city, built as a group project in the Department of Electrical and Electronic Engineering, University of Chittagong. Seven independent systems each run on their own microcontroller and perform one real task: monitoring the weather, detecting smoke and gas, controlling access to a building, managing a car park, handling waste, running traffic lights and tracking the sun. The weather station's ESP32 also acts as the city's central hub. It collects data from the other systems and uploads everything to a cloud database, from which a public web dashboard shows the whole city live.



Figure 1. The completed ORBIS model (top view).

1.1 Project summary

Table 1. Project details

Item	Details
Project	ORBIS – Smart Mini City
Institution	Department of EEE, University of Chittagong
Completed	June 2026; presented at the project viva
Controllers	2 × ESP32, 5 × Arduino UNO
Cloud services	Cloudflare Workers, Supabase, Vercel
Live dashboard	orbis-dashboard-amber.vercel.app

1.2 Team

Table 2. Team members

Student ID	Name	Role
23702051	Nabiha Nowrin	Gas and fire monitor; RFID door lock
24702028	Prattasha Majumder	Design and research
24702031	Afrin Mostafa Riya	Web dashboard
24702035	Safia Selim	Design
24702045	Sabrin Sultana	Smart dustbin
24702050	Syeda Muntaha Najah	Smart parking
24702040	Minhaj Uddin Samrat	Solar tracker

1.3 My contribution

I (Nabiha Nowrin) built the **gas and fire monitor** (Section 3) and the **RFID door lock** (Section 4), including the fire-alert link between the two systems.

1.4 Systems at a glance

Table 3. The seven ORBIS systems

Section	System	Controller	Main function
3	Gas & Fire Monitor	Arduino UNO	Detects smoke and gas; starts a sprinkler pump; signals the door to open
4	RFID Door Lock	ESP32	Card access to the building; shows flat and time; opens in a fire
5	Smart Parking	Arduino UNO	Counts 4 slots; automatic barrier gate; full alarm
6	Smart Dustbin	Arduino UNO	Touch-free lid; measures fill level; full alarm
7	Traffic Lights	Arduino UNO	Two-phase four-way junction with countdown displays
8	Solar Tracker	Arduino UNO	PID control turns the panel toward the light
9	Weather Station + Hub	ESP32	Temperature, humidity, heat index and light; collects and uploads all data

2 System Architecture

Every system works on its own, with its own sensors, outputs and display. In addition, each one reports its state to the hub, so the whole city can be monitored from one place. The figure below shows how data flows from the sensors to the web dashboard.

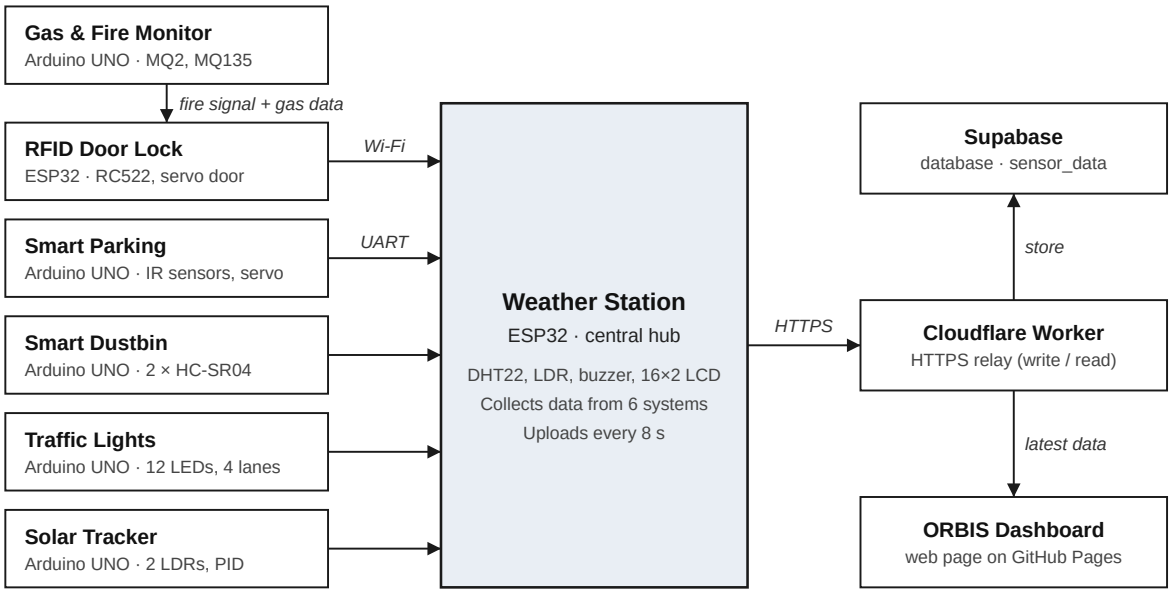


Figure 2. ORBIS system architecture. Five systems connect directly to the hub; the gas monitor reaches the hub through the RFID door's ESP32.

Table 4. Data links between the systems

From	To	Link	Example message
Gas monitor	RFID ESP32	UART 9600 baud + signal wire	`MQ2:350,MQ135:280,STATUS:SAFE,PUMP:OFF`
RFID ESP32	Hub ESP32	Wi-Fi, HTTP POST to `/sensordata`	door, uid, flat, mq2, mq135, gas, pump
Smart parking	Hub GPIO 16	UART 9600 baud	`GATE=1,SLOTS=3,STATUS=OPEN`
Smart dustbin	Hub GPIO 13	UART 9600 baud	`FILL=40,DEPTH=6,LID=CLOSED,STATUS=OK`
Traffic lights	Hub GPIO 14	Software serial 9600 baud	`PHASE=1,DIR=N + S,TIME=7,STATE=GREEN,TOTAL=10`
Solar tracker	Hub GPIO 25	Software serial 9600 baud	`L:812,R:705,Err:19.0,Ang:96.4`
Hub ESP32	Cloudflare Worker	HTTPS POST every 8 s	One JSON object with all readings
Dashboard	Cloudflare Worker	HTTPS GET	Newest row from the database

2.1 Fire alert across three devices

The most important interaction in the city is the fire response, which involves three separate boards:

1. The MQ2 or MQ135 reading on the gas monitor reaches the danger threshold (500).
2. The gas monitor switches its relay, which runs the water-pump sprinkler, and shows *FIRE ALERT* on its LCD.
3. At the same time it drives its signal wire HIGH, and the RFID door's ESP32 opens the door for evacuation.
4. The danger status reaches the hub, which sounds a continuous alarm and shows *EVACUATE NOW*.
5. The web dashboard flashes red and shows a fire-alert pop-up.

2.2 Communication links

The boards use three kinds of connection: wired serial (UART) lines between the Arduinos and the hub, Wi-Fi between the two ESP32 boards, and HTTPS between the hub and the cloud. Each serial message is a single line of plain text with comma-separated fields, which the receiving board splits into values.

3 Gas and Fire Monitor

The gas and fire monitor is an Arduino UNO with two gas sensors. When smoke or gas reaches a dangerous level, it starts a water-pump sprinkler, shows a red warning and signals the RFID door to open so that residents can leave.



Figure 3. Residential building with the gas monitor, water tank and sprinkler pipe.

3.1 Function

The MQ2 sensor responds to smoke and flammable gases such as LPG; the MQ135 responds to air pollutants and toxic gases. Every 0.3 s both are read, and the higher reading decides the alarm level, as shown below.

Table 5. Gas monitor alarm levels (ADC reading 0–1023)

Level	Reading	LED	Pump	LCD message
Safe	below 350	Green	Off	Air is Clean / All Clear
Warning	350–499	Yellow	Off	Warning! / Moderate Level
Danger	500 and above	Red	On	FIRE ALERT!! / Evacuate Now!

3.2 Operation

1. **Sensing.** Each MQ sensor outputs an analog voltage that rises with gas concentration; the UNO's ADC converts it to a value from 0 to 1023.
2. **Classification.** The larger of the two values is compared with the thresholds 350 and 500.
3. **Sprinkler.** At the danger level the relay is switched on, which powers the water pump. The relay module is active-low, so the pin is driven LOW to start the pump.
4. **Door release.** At the danger level pin D3 is set HIGH. This line is wired to the RFID door's ESP32, which opens the door.
5. **Reporting.** Once per second a status line is sent over serial to the RFID ESP32, which forwards it to the hub over Wi-Fi.
6. **Display.** A push button cycles three LCD screens: raw sensor values, status, and the alert message.

3.3 Components

Table 6. Gas and fire monitor components

Component	Qty	Purpose
Arduino UNO	1	Controller
MQ2 gas sensor	1	Smoke and flammable gas
MQ135 gas sensor	1	Air quality and toxic gas
5 V relay module	1	Switches the pump
Mini water pump, tubing and tank	1	Sprinkler
LEDs (green, yellow, red)	3	Status
16×2 LCD with I2C adapter	1	Display
Push button	1	Screen selection

3.4 Pin connections

Table 7. Gas monitor pin assignment

Signal	Arduino UNO pin
MQ2 analog output	A0
MQ135 analog output	A1
Green / yellow / red LED	D8 / D7 / D6
Push button (internal pull-up)	D2
Relay input	D4
Fire signal to RFID ESP32	D3 → GPIO 16
Serial data to RFID ESP32	D1 (TX) → GPIO 17
LCD SDA / SCL (I2C, address 0x27)	A4 / A5

3.5 Software

Arduino IDE. Libraries: Wire, LiquidCrystal_I2C.

3.6 Source code

Listing 1. Gas and fire monitor firmware (Arduino UNO)

```
/*
  GAS SENSOR — Arduino UNO
  Sends data over Serial to RFID ESP32
  Format: MQ2:350,MQ135:280,STATUS:SAFE,PUMP:OFF\n

  PINS
  ────
  MQ2      → A0
  MQ135    → A1
  GREEN    → D8
  YELLOW   → D7
  RED      → D6
  BUTTON   → D2
  RELAY    → D4
```

```
  SIGNAL   → D3  → RFID ESP32 GPIO16
  TX       → D1  → RFID ESP32 GPIO17
*/

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

#define SIGNAL_PIN 3
#define MQ2_PIN    A0
#define MQ135_PIN  A1
#define GREEN_LED  8
#define YELLOW_LED 7
#define RED_LED    6
#define BUTTON_PIN 2
#define RELAY_PIN  4

#define LOW_THRESHOLD 350
#define MID_THRESHOLD 500

int currentScreen = 0;
bool lastButtonState = HIGH;
unsigned long lastSerial = 0;
#define SERIAL_INTERVAL 1000UL

void setup() {
  pinMode(SIGNAL_PIN, OUTPUT);
  digitalWrite(SIGNAL_PIN, LOW);
  Serial.begin(9600);
  pinMode(GREEN_LED,  OUTPUT);
  pinMode(YELLOW_LED, OUTPUT);
  pinMode(RED_LED,    OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(RELAY_PIN,  OUTPUT);
  digitalWrite(RELAY_PIN, HIGH);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0,0); lcd.print("Gas Monitor");
  lcd.setCursor(0,1); lcd.print("Starting up...");
  delay(2000);
  lcd.clear();
}

void setLED(int level) {
  digitalWrite(GREEN_LED,  LOW);
  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(RED_LED,    LOW);
  if(level==1) digitalWrite(GREEN_LED,  HIGH);
  else if(level==2) digitalWrite(YELLOW_LED, HIGH);
  else if(level==3) digitalWrite(RED_LED,    HIGH);
}

void checkButton() {
  bool cur = digitalRead(BUTTON_PIN);
  if(lastButtonState==HIGH && cur==LOW){
    currentScreen=(currentScreen+1)%3;
    lcd.clear(); delay(200);
  }
  lastButtonState=cur;
}

void loop() {
  int mq2    = analogRead(MQ2_PIN);
  int mq135  = analogRead(MQ135_PIN);
  int maxVal = max(mq2, mq135);

  String status;
  String line1, line2;
  int level;
```

```
  bool pumpOn;

  if(maxVal < LOW_THRESHOLD){
    status="SAFE"; line1="Air is Clean"; line2="All Clear"; level=1; pumpOn=false;
  } else if(maxVal < MID_THRESHOLD){
    status="WARNING"; line1="Warning!"; line2="Moderate Level"; level=2; pumpOn=false;
  } else {
    status="DANGER"; line1="FIRE ALERT!!"; line2="Evacuate Now!"; level=3; pumpOn=true;
  }

  digitalWrite(SIGNAL_PIN, level==3 ? HIGH : LOW);
  digitalWrite(RELAY_PIN,  pumpOn   ? LOW  : HIGH);
  setLED(level);
  checkButton();

  // LCD screens
  if(currentScreen==0){
    lcd.setCursor(0,0); lcd.print("Smoke: "); lcd.print(mq2);   lcd.print("     ");
    lcd.setCursor(0,1); lcd.print("Toxin: "); lcd.print(mq135); lcd.print("     ");
  } else if(currentScreen==1){
    lcd.setCursor(0,0); lcd.print("Status: "); lcd.print(status); lcd.print("     ");
    lcd.setCursor(0,1); lcd.print(line2); lcd.print("     ");
  } else {
    lcd.setCursor(0,0); lcd.print(line1); lcd.print("     ");
    lcd.setCursor(0,1); lcd.print(line2); lcd.print("     ");
  }

  // Send to RFID ESP32 every 1 second
  if(millis()-lastSerial >= SERIAL_INTERVAL){
    lastSerial=millis();
    Serial.print("MQ2:"); Serial.print(mq2);
    Serial.print(",MQ135:"); Serial.print(mq135);
    Serial.print(",STATUS:"); Serial.print(status);
    Serial.print(",PUMP:"); Serial.println(pumpOn ? "ON" : "OFF");
  }

  delay(300);
}
```

4 RFID Door Lock

The RFID door lock is an ESP32-based access-control system for the city's residential building. Residents tap an RFID card at the gate; registered cards open the door and the OLED display shows which flat entered and at what time. Unknown cards are refused. In the complete ORBIS setup the door was also linked to the gas monitor (Section 3), so that it opens automatically when a fire is detected.

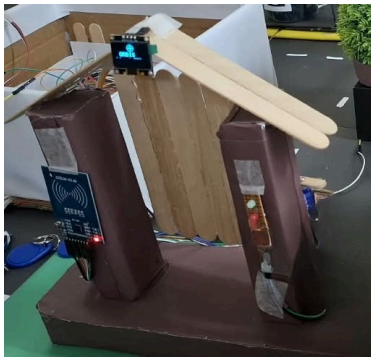


Figure 4. RFID gate with the RC522 reader, OLED display and door servo.

4.1 Function

- Reads the unique identifier (UID) of each card with an RC522 RFID reader.
- Recognises two registered cards, one for Flat 4A and one for Flat 4B.
- **Access granted:** green LED on, the servo opens the door for 2 s, and the OLED shows the flat name, *GRANTED* and the time; the door then closes and the display shows *CLOSED*.
- **Access denied:** red LED on for 2 s and the OLED shows *ACCESS DENIED!* with the time.
- **Standby:** the OLED shows *RFID LOCK – Scan card* and a live clock, refreshed every 10 s.

4.2 Operation

1. **Clock.** At start-up the ESP32 connects to Wi-Fi and synchronises its clock with an internet time server (NTP) set to Bangladesh time (UTC+6), so every entry is shown with the correct time.
2. **Reading a card.** The RC522 communicates with the ESP32 over the SPI bus. When a card is held near the reader, its 4-byte UID is read and printed to the serial monitor.
3. **Checking the card.** The UID is compared byte by byte with the stored UIDs of the two registered cards.
4. **Opening the door.** On a match the servo turns from 0° (closed) to 90° (open), waits 2 s and returns to 0°.
5. **Ending the read.** The card is halted and the reader's encryption is stopped, so the same tap is not read twice.

4.3 Components

Table 8. RFID door lock components

Component	Qty	Purpose
ESP32 development board	1	Controller with Wi-Fi
RC522 RFID reader with cards/tags	1 set	Card reading
Servo motor	1	Door actuator
0.96" OLED display (SSD1306, I2C)	1	Status and clock
LEDs (green, red)	2	Access granted / denied

4.4 Pin connections

Table 9. RFID door lock pin assignment

Signal	ESP32 pin
RC522 SDA (SS)	GPIO 5
RC522 SCK	GPIO 18
RC522 MISO	GPIO 19
RC522 MOSI	GPIO 23
RC522 RST	GPIO 4
Servo signal	GPIO 13
Green LED	GPIO 26
Red LED	GPIO 27
OLED SDA / SCL (I2C, address 0x3C)	GPIO 21 / GPIO 22

4.5 Software

Arduino IDE with the ESP32 board package. Libraries: SPI, MFRC522, ESP32Servo, Wire, Adafruit GFX, Adafruit SSD1306, WiFi and time (ESP32 core).

4.6 Source code

The Wi-Fi network name and password are replaced with placeholders. The firmware was revised several times during the project, including a move to a replacement ESP32 board; this listing is the access-control version.

Listing 2. RFID door lock firmware (ESP32)

```
#include <SPI.h>
#include <MFRC522.h>
#include <ESP32Servo.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <WiFi.h>
#include <time.h>

// WiFi credentials
const char* ssid = "WIFI_NAME";
const char* password = "WIFI_PASSWORD";

// NTP settings
const char* ntpServer = "pool.ntp.org";
const long gmtOffset_sec = 21600;  // Bangladesh = UTC+6
const int daylightOffset_sec = 0;

// Pins
#define RST_PIN 4
#define SS_PIN 5
#define SERVO_PIN 13
#define GREEN_LED 26
#define RED_LED 27

// OLED
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

```

MFRC522 rfid(SS_PIN, RST_PIN);
Servo doorServo;

// Card UIDs
byte card4A[4] = {0xA9, 0x91, 0x31, 0x03};  // Flat 4A
byte card4B[4] = {0xA9, 0x7E, 0x46, 0x03};  // Flat 4B

bool checkUID(byte *uid, byte *authorizedUID) {
  for (byte i = 0; i < 4; i++) {
    if (uid[i] != authorizedUID[i]) return false;
  }
  return true;
}

String getTime() {
  struct tm timeinfo;
  if (!getLocalTime(&timeinfo)) return "No Time";
  char timeStr[20];
  strftime(timeStr, sizeof(timeStr), "%H:%M:%S", &timeinfo);
  return String(timeStr);
}

void showStandby() {
  String currentTime = getTime();
  display.clearDisplay();
  display.setTextSize(2);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);
  display.println("RFID LOCK");
  display.setCursor(0, 25);
  display.println("Scan card");
  display.setCursor(0, 48);
  display.println(currentTime);
  display.display();
}

void openDoor(String flatName) {
  digitalWrite(GREEN_LED, HIGH);
  digitalWrite(RED_LED, LOW);
  doorServo.write(90);

  String currentTime = getTime();
  display.clearDisplay();
  display.setTextSize(2);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);
  display.println(flatName);
  display.setCursor(0, 25);
  display.println("GRANTED");
  display.setCursor(0, 48);
  display.println(currentTime);
  display.display();

  delay(2000);

  doorServo.write(0);
  digitalWrite(GREEN_LED, LOW);

  display.clearDisplay();
  display.setTextSize(2);
  display.setCursor(0, 0);
  display.println(flatName);
  display.setCursor(0, 25);
  display.println("CLOSED");
  display.setCursor(0, 48);
  display.println(getTime());
  display.display();

```

```
  delay(1000);
  showStandby();
}

void denyAccess() {
  digitalWrite(RED_LED, HIGH);
  digitalWrite(GREEN_LED, LOW);

  String currentTime = getTime();
  display.clearDisplay();
  display.setTextSize(2);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);
  display.println("ACCESS");
  display.setCursor(0, 25);
  display.println("DENIED!");
  display.setCursor(0, 48);
  display.println(currentTime);
  display.display();

  delay(2000);
  digitalWrite(RED_LED, LOW);
  showStandby();
}

void setup() {
  Serial.begin(115200);

  // Pins
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);

  // Servo
  doorServo.attach(SERVO_PIN);
  doorServo.write(0);
  delay(500);

  // OLED
  Wire.begin(21, 22);
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED not found!");
    while (true);
  }

  // Connecting screen
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);
  display.println("Connecting WiFi...");
  display.display();

  // WiFi
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("WiFi connected!");

  // NTP time sync
  configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
  delay(2000);

  // RFID
  SPI.begin(18, 19, 23, 5);
  rfid.PCD_Init();
```

```
169
170   showStandby();
171   Serial.println("RFID Door Lock Ready!");
172 }
173
174 void loop() {
175   // Update time on standby every 10 seconds
176   static unsigned long lastUpdate = 0;
177   if (millis() - lastUpdate > 10000) {
178     showStandby();
179     lastUpdate = millis();
180   }
181
182   if (!rfid.PICC_IsNewCardPresent()) return;
183   if (!rfid.PICC_ReadCardSerial()) return;
184
185   byte *uid = rfid.uid.uidByte;
186
187   Serial.print("Card UID:");
188   for (byte i = 0; i < 4; i++) {
189     Serial.print(" ");
190     Serial.print(uid[i], HEX);
191   }
192   Serial.println();
193
194   if (checkUID(uid, card4A)) {
195     openDoor("Flat 4A");
196   } else if (checkUID(uid, card4B)) {
197     openDoor("Flat 4B");
198   } else {
199     denyAccess();
200   }
201
202   rfid.PICC_HaltA();
203   rfid.PCD_StopCrypto1();
204   delay(1000);
205 }
```

5 Smart Parking

The smart parking system is a four-slot car park with an automatic barrier gate. It counts vehicles entering and leaving, shows the number of free slots, and refuses entry with an audible alarm when the car park is full.

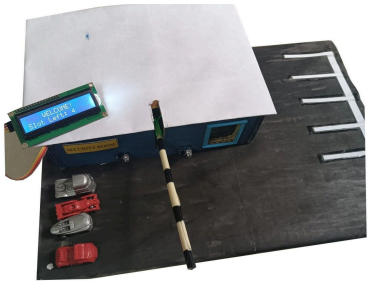


Figure 5. Parking area with the security room, barrier gate and LCD.

5.1 Function

- An IR sensor at the entrance detects an arriving car. If a slot is free, the barrier lifts, the buzzer beeps once and the LCD shows *GATE OPENING – WELCOME IN!*
- A second IR sensor detects a departing car and lifts the barrier with the message *GOODBYE!*
- The barrier closes automatically after 3 s and the slot count is updated.
- When all four slots are occupied, an arriving car triggers three short beeps, the red LED and the message *SORRY :(Slots Are Full*. The alarm then waits 3 s before it can repeat.
- Green LED: space available. Red LED: full. Status is sent to the hub every 0.5 s.

5.2 Operation

1. **Detection.** The IR obstacle sensors output LOW when a car blocks the beam.
2. **Counting.** The variable `slots` holds the number of parked cars (0–4). It is updated only after the gate closes, so each car is counted once.
3. **Gate control.** The servo moves to 0° to open and 90° to close, and is detached between movements to prevent jitter.
4. **Non-blocking timing.** Gate, alarm and buzzer timing use `millis()` rather than long delays, so the sensors are read continuously.
5. **Testing.** Sending `E` over the serial monitor simulates a car entering; `R` resets the count.

5.3 Components

Table 10. Smart parking components

Component	Qty	Purpose
Arduino UNO	1	Controller
IR obstacle sensor	2	Entry and exit detection
Servo motor	1	Barrier gate
16×2 LCD with I2C adapter	1	Display
Buzzer	1	Signals and full alarm
LEDs (green, red)	2	Availability

5.4 Pin connections

Table 11. Smart parking pin assignment

Signal	Arduino pin
Entry IR sensor	D2
Exit IR sensor	D3
Green / red LED	D6 / D7
Buzzer	D8
Servo signal	D9
LCD SDA / SCL (I2C, address 0x27)	A4 / A5
Serial data to hub	D1 (TX) → GPIO 16

5.5 Software

Arduino IDE. Libraries: Wire, Servo, LiquidCrystal_I2C.

5.6 Source code

Listing 3. Smart parking firmware (Arduino UNO)

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo myservo;

const int IR_GATE   = 2;
const int IR_SLOT   = 3;
const int BUZZER    = 8;
const int GREEN_LED = 6;
const int RED_LED   = 7;
const int TOTAL     = 4;
bool alerting = false;

unsigned long gateOpenTime = 0;
unsigned long fullAlertTime = 0;
unsigned long buzzerTime = 0;
bool gateOpen = false;
bool fullAlertShowing = false;
bool buzzerOn2 = false;
int buzzerCount = 0;
unsigned long lastSerial = 0;
#define GATE_OPEN_TIME 3000UL
#define FULL_ALERT_TIME 2000UL

unsigned long lastFullAlert = 0;
#define FULL_ALERT_COOLDOWN 3000UL

int slots = 0;  // 0 = empty, 4 = full
int flag1 = 0;
int flag2 = 0;

void showDisplay() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("    WELCOME!    ");
  lcd.setCursor(0, 1);
```

```
  if (slots == TOTAL) {
    lcd.print(" Slots Are Full ");
  } else {
    lcd.print("Slot Left: ");
    lcd.print(TOTAL - slots);
    lcd.print("   ");
  }
}

void fullAlert() {
  alerting = true;
  digitalWrite(GREEN_LED, LOW);
  for (int i = 0; i < 3; i++) {
    digitalWrite(RED_LED, HIGH);
    digitalWrite(BUZZER, HIGH);
    delay(150);
    digitalWrite(RED_LED, LOW);
    digitalWrite(BUZZER, LOW);
    delay(100);
  }
  digitalWrite(RED_LED, HIGH);
  alerting = false;
  delay(500);
}

void setup() {
  Serial.begin(9600);
  pinMode(IR_GATE,   INPUT_PULLUP);
  pinMode(IR_SLOT,   INPUT_PULLUP);
  pinMode(BUZZER,    OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED,   OUTPUT);

  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0); lcd.print("     ARDUINO    ");
  lcd.setCursor(0, 1); lcd.print(" PARKING SYSTEM ");
  delay(2000);
  lcd.clear();

myservo.attach(9);
myservo.write(90);
delay(800);
myservo.detach();

  showDisplay();
}

void loop() {
  unsigned long now = millis();
  int g = digitalRead(IR_GATE);
  int s = digitalRead(IR_SLOT);

  // Serial status to hub every 500 ms
  if (now - lastSerial >= 500) {
    lastSerial = now;
    Serial.print("GATE="); Serial.print(g);
    Serial.print(",SLOTS="); Serial.print(TOTAL - slots);
    Serial.print(",STATUS="); Serial.println(slots == TOTAL ? "FULL" : "OPEN");
  }

  // Manual serial commands
  if (Serial.available()) {
    char c = Serial.read();
    if (c == 'E') { if (slots < TOTAL) { slots++; showDisplay(); } }
    if (c == 'R') { slots = 0; showDisplay(); }
  }

  // Buzzer non-blocking
```

```
  if (buzzerOn2 && now - buzzerTime >= 150) {
    buzzerTime = now;
    buzzerCount++;
    if (buzzerCount % 2 == 0) digitalWrite(BUZZER, LOW);
    else digitalWrite(BUZZER, HIGH);
    if (buzzerCount >= 6) {
      buzzerOn2 = false;
      buzzerCount = 0;
      digitalWrite(BUZZER, LOW);
      digitalWrite(RED_LED, HIGH);
    }
  }

  // LED logic
  if (!buzzerOn2) {
    if (slots == TOTAL) {
      digitalWrite(RED_LED, HIGH);
      digitalWrite(GREEN_LED, LOW);
    } else {
      digitalWrite(RED_LED, LOW);
      digitalWrite(GREEN_LED, HIGH);
    }
  }

  // Close gate after time
    if (gateOpen && now - gateOpenTime >= GATE_OPEN_TIME) {
    gateOpen = false;
    myservo.attach(9);
    myservo.write(90);
    delay(800);
    myservo.detach();
    if (flag1 == 1) {
      if (slots < TOTAL) slots++;
      flag1 = 0;
      showDisplay();
    }
    if (flag2 == 1) {
      if (slots > 0) slots--;
      flag2 = 0;
      showDisplay();
    }
  }

  // Full alert timeout
  if (fullAlertShowing && now - fullAlertTime >= FULL_ALERT_TIME) {
    fullAlertShowing = false;
    showDisplay();
  }

  // CAR ENTERING
  if (g == LOW && flag1 == 0 && flag2 == 0 && !gateOpen) {
    if (slots < TOTAL) {
      flag1 = 1;
      gateOpen = true;
      gateOpenTime = now;
      myservo.attach(9);
      myservo.write(0);
      digitalWrite(BUZZER, HIGH);
      delay(80);
      digitalWrite(BUZZER, LOW);
      lcd.setCursor(0, 0); lcd.print("  GATE OPENING  ");
      lcd.setCursor(0, 1); lcd.print("  WELCOME IN!   ");
    } else {
      if (now - lastFullAlert >= FULL_ALERT_COOLDOWN) {
        lastFullAlert = now;
        // Beep immediately RIGHT NOW
        digitalWrite(BUZZER, HIGH);
        delay(150);
        digitalWrite(BUZZER, LOW);
```

```
        delay(100);
        digitalWrite(BUZZER, HIGH);
        delay(150);
        digitalWrite(BUZZER, LOW);
        delay(100);
        digitalWrite(BUZZER, HIGH);
        delay(150);
        digitalWrite(BUZZER, LOW);
        fullAlertShowing = true;
        fullAlertTime = now;
        lcd.clear();
        lcd.setCursor(0, 0); lcd.print("    SORRY :(    ");
        lcd.setCursor(0, 1); lcd.print(" Slots Are Full ");
      }
    }
  }

  // CAR EXITING
  if (s == LOW && flag2 == 0 && flag1 == 0 && !gateOpen) {
    flag2 = 1;
    gateOpen = true;
    gateOpenTime = now;
      myservo.attach(9);
      myservo.write(0);
      digitalWrite(BUZZER, HIGH);
      delay(80);
      digitalWrite(BUZZER, LOW);
      lcd.setCursor(0, 0); lcd.print("  GATE OPENING  ");
      lcd.setCursor(0, 1); lcd.print("   GOODBYE!     ");
  }
}
```

6 Smart Dustbin

The smart dustbin has a touch-free lid that opens when a hand approaches and closes after it leaves. A second sensor inside the bin measures the fill level and raises an alarm when the bin needs emptying.

6.1 Function

- A hand within 15 cm of the outer sensor opens the lid. When no hand has been detected for 2 s, the distance is checked again and the lid closes.
- While the lid is closed, the inner sensor measures the distance to the waste and converts it to a fill percentage.
- At 90 % or more: red LED, *Status: FULL!* on the LCD and two long beeps (once per filling). Below 90 %: green LED.
- Fill percentage, depth, lid state and status are sent to the hub.

6.2 Operation

The HC-SR04 ultrasonic sensor emits a sound pulse and measures the echo time t in microseconds. Since sound travels about 0.034 cm/µs and the pulse covers the distance twice, the distance is

$$d = t \,/\, 58.82 \ \text{cm} \tag{1}$$

Each distance is the mean of four pulses, and the fill measurement averages ten such values to reject stray echoes. With the bin reading 9 cm when empty and 3 cm when full, the fill level is

$$\text{Fill}\ (\%) = (9 - d) \,/\, (9 - 3) \times 100 \tag{2}$$

The servo moves the lid in 5° steps between 90° (closed) and 0° (open) for smooth motion, and the fill level is measured only while the lid is closed so that the lid or a hand cannot affect the reading.

6.3 Components

Table 12. Smart dustbin components

Component	Qty	Purpose
Arduino UNO	1	Controller
HC-SR04 ultrasonic sensor	2	Hand detection and fill level
Servo motor	1	Lid actuator
16×2 LCD with I2C adapter	1	Display
Buzzer	1	Full alarm
LEDs (green, red)	2	Status

6.4 Pin connections

Table 13. Smart dustbin pin assignment

Signal	Arduino pin
Lid sensor TRIG / ECHO	D2 / D3
Depth sensor TRIG / ECHO	D4 / D5
Red LED	D7
Buzzer	D8
Servo signal	D9
Green LED	D10
LCD SDA / SCL (I2C, address 0x27)	A4 / A5
Serial data to hub	D1 (TX) → GPIO 13

6.5 Software

Arduino IDE. Libraries: Wire, Servo, LiquidCrystal_I2C.

6.6 Source code

Listing 4. Smart dustbin firmware (Arduino UNO)

```
#include <Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);
Servo lid;

// Lid sensor
int trig1 = 2;
int echo1 = 3;

// Depth sensor
int trig2 = 4;
int echo2 = 5;

int ledRed   = 7;
int ledGreen = 10;
int buzzer   = 8;
int servoPin = 9;

bool isLidOpen        = false;
bool hasBeepedForFull = false;

const int EMPTY_DIST = 9;
const int FULL_DIST  = 3;

// ── Get filtered distance ──────────────────────────────────────────
long getFilteredDistance(int trigPin, int echoPin) {
  long totalDistance = 0;
  int samples = 4;
  for (int i = 0; i < samples; i++) {
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    long duration = pulseIn(echoPin, HIGH, 25000);
```

```
    long distance = duration / 58.82;
    if (distance <= 0 || distance > 200) distance = 200;
    totalDistance += distance;
    delay(5);
  }
  return totalDistance / samples;
}

// ── Fill % calculation ──────────────────────────────────────────
int getFillPercent(long distance) {
  if (distance >= EMPTY_DIST) return 0;
  if (distance <= FULL_DIST)  return 100;
  return (int)((float)(EMPTY_DIST - distance) / (EMPTY_DIST - FULL_DIST) * 100);
}

// ── Buzzer ──────────────────────────────────────────────────────
void beepTwice() {
  digitalWrite(buzzer, HIGH);
  delay(800);
  digitalWrite(buzzer, LOW);
  delay(300);
  digitalWrite(buzzer, HIGH);
  delay(800);
  digitalWrite(buzzer, LOW);
}

// ── Lid control ─────────────────────────────────────────────────
void openLid() {
  for (int a = 90; a >= 0; a -= 5) {
    lid.write(a);
    delay(10);
  }
  isLidOpen = true;
}

void closeLid() {
  for (int a = 0; a <= 90; a += 5) {
    lid.write(a);
    delay(10);
  }
  isLidOpen = false;
  delay(1000);
}

// ── Setup ───────────────────────────────────────────────────────
void setup() {
  Serial.begin(9600);
  lcd.init();
  lcd.backlight();

  lid.attach(servoPin);

  pinMode(trig1, OUTPUT);
  pinMode(echo1, INPUT);
  pinMode(trig2, OUTPUT);
  pinMode(echo2, INPUT);
  pinMode(buzzer,   OUTPUT);
  pinMode(ledRed,   OUTPUT);
  pinMode(ledGreen, OUTPUT);

  lid.write(90);
  delay(500);

  // welcome message
  lcd.setCursor(0, 0);
  lcd.print("  Smart Bin  ");
  lcd.setCursor(0, 1);
  lcd.print("  Starting...  ");
  delay(2000);
```

```
  lcd.clear();
}

// ── Main loop ──────────────────────────────────────────────────────────
void loop() {

  // — Lid sensor —
  long outside = getFilteredDistance(trig1, echo1);
  bool handPresent = (outside >= 1 && outside <= 15);

  // hand present → open lid
  if (handPresent && !isLidOpen) {
    openLid();
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Lid: OPEN");
  }

  // no hand → close lid
  if (!handPresent && isLidOpen) {
    delay(2000);
    long recheck = getFilteredDistance(trig1, echo1);
    if (recheck > 15) {
      closeLid();
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("Lid: CLOSED");
    }
  }

// — Depth + LED + Buzzer — only when lid is closed —
  if (!isLidOpen) {
    long total = 0;
    for (int i = 0; i < 10; i++) {
      total += getFilteredDistance(trig2, echo2);
      delay(30);
    }
    long depth = total / 10;
    int fillPct = getFillPercent(depth);

    if (fillPct >= 90) {
      digitalWrite(ledRed,   HIGH);
      digitalWrite(ledGreen, LOW);
      if (!hasBeepedForFull) {
        beepTwice();
        hasBeepedForFull = true;
      }
    } else {
      digitalWrite(ledRed,   LOW);
      digitalWrite(ledGreen, HIGH);
      hasBeepedForFull = false;
    }

    lcd.setCursor(0, 0);
    lcd.print("Fill: ");
    lcd.print(fillPct);
    lcd.print("%   ");

    lcd.setCursor(0, 1);
    if (fillPct >= 90) {
      lcd.print("Status: FULL!   ");
    } else {
      lcd.print("Status: OK      ");
    }

    // — Serial —
    Serial.print("Lid: "); Serial.print(isLidOpen ? "OPEN" : "CLOSED");
    Serial.print(" | Depth: "); Serial.print(depth); Serial.print("cm");
    Serial.print(" | Fill: "); Serial.print(fillPct); Serial.print("%");
```

```
    Serial.print(" | Status: "); Serial.println(fillPct >= 90 ? "FULL" : "EMPTY");

    Serial.print("FILL="); Serial.print(fillPct);
    Serial.print(",DEPTH="); Serial.print(depth);
    Serial.print(",LID=CLOSED");
    Serial.print(",STATUS="); Serial.println(fillPct >= 90 ? "FULL" : "OK");

  } else {
    // Lid is open — still print serial
    Serial.println("Lid: OPEN | Depth: --cm | Fill: --%  | Status: --");
    Serial.println("FILL=0,DEPTH=0,LID=OPEN,STATUS=OK");
  }

  delay(100);
}
```

7 Traffic Lights

The traffic light system runs the signals of a four-way junction. Opposite approaches receive green together, every change passes through amber and a short all-red interval, and a 16×2 LCD counts down the remaining time. A second display, an OLED showing the direction and a shrinking progress bar, was built and demonstrated working, but was left off the final model for the finished look.

7.1 Function

Four signal heads (12 LEDs) follow a fixed two-phase cycle of 28 s, shown below. An LCD shows the active phase and the seconds remaining; the OLED shows the direction in large text with a shrinking progress bar.

Table 14. Signal timing for one cycle

Interval	North + South	East + West	Duration
Phase 1	Green	Red	10 s
Clearance	Amber	Red	3 s
All-red	Red	Red	1 s
Phase 2	Red	Green	10 s
Clearance	Red	Amber	3 s
All-red	Red	Red	1 s

7.2 Operation

1. **Lane table.** The signal pins are stored in a 4 × 3 array (one row per approach: red, amber, green), so any approach can be set with a single function call.
2. **Phase table.** Each phase lists its two green approaches, its duration and its display text; the main loop runs phase 1 and phase 2 repeatedly.
3. **All-red interval.** Every approach is set to red before any approach turns green, so conflicting directions never move together.
4. **Countdown bar.** The bar width is computed each second as `map(secLeft, 0, totalSec, 0, 124)`, and the OLED is redrawn with the new bar.

7.3 Components

Table 15. Traffic light components

Component	Qty	Purpose
Arduino UNO	1	Controller
LEDs (red, amber, green)	12	Four signal heads
16×2 LCD with I2C adapter	1	Phase and time remaining
0.96" OLED (SSD1306, I2C)	1	Direction and countdown bar (demonstrated separately)

7.4 Pin connections

Table 16. Traffic signal pin assignment

Approach	Red	Amber	Green
North	D2	D3	D4
South	D5	D6	D7
East	D8	D9	D10
West	D11	D12	D13

The LCD (0x27) and OLED (0x3C) share the I2C bus on A4/A5.

7.5 Software

Arduino IDE. Libraries: Wire, LiquidCrystal_I2C, Adafruit GFX, Adafruit SSD1306.

7.6 Source code

Listing 5. Traffic light firmware (Arduino UNO)

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define OLED_WIDTH  128
#define OLED_HEIGHT  64
#define OLED_RESET   -1

LiquidCrystal_I2C lcd(0x27, 16, 2);
Adafruit_SSD1306 oled(OLED_WIDTH, OLED_HEIGHT, &Wire, OLED_RESET);

const int lanes[4][3] = {
  {2,  3,  4},
  {5,  6,  7},
  {8,  9,  10},
  {11, 12, 13},
};

struct Phase {
  int greenLanes[2];
  int durationMs;
  const char* lcdLabel;
  const char* oledDir;
};

Phase phases[2] = {
  {{0, 1}, 10000, "Phase 1: N+S GO", "N + S"},
  {{2, 3}, 10000, "Phase 2: E+W GO", "E + W"},
};

const int AMBER_MS   = 3000;
const int ALL_RED_MS = 1000;

void allRed() {
  for (int i = 0; i < 4; i++) {
    digitalWrite(lanes[i][0], HIGH);
    digitalWrite(lanes[i][1], LOW);
    digitalWrite(lanes[i][2], LOW);
  }
}
```

```
 42 void setGreen(int i) { digitalWrite(lanes[i][0],LOW);  digitalWrite(lanes[i][1],LOW);
digitalWrite(lanes[i][2],HIGH); }
 43 void setAmber(int i) { digitalWrite(lanes[i][0],LOW);  digitalWrite(lanes[i][1],HIGH);
digitalWrite(lanes[i][2],LOW);  }
 44
 45 void updateLCD(int phaseNum, int secLeft, bool amber) {
 46   lcd.clear();
 47   lcd.setCursor(0, 0);
 48   if (amber) lcd.print("AMBER-clearing  ");
 49   else       lcd.print(phases[phaseNum].lcdLabel);
 50   lcd.setCursor(0, 1);
 51   lcd.print("Time left: ");
 52   lcd.print(secLeft);
 53   lcd.print("s  ");
 54 }
 55
 56 void updateOLED(int phaseNum, int secLeft, int totalSec, bool amber) {
 57   oled.clearDisplay();
 58
 59   oled.setTextSize(1);
 60   oled.setTextColor(SSD1306_WHITE);
 61   oled.setCursor(0, 0);
 62   oled.print(amber ? "AMBER" : (phaseNum == 0 ? "PHASE 1" : "PHASE 2"));
 63
 64   oled.setTextSize(2);
 65   oled.setCursor(0, 14);
 66   oled.print(amber ? "CLEAR " : phases[phaseNum].oledDir);
 67
 68   oled.setTextSize(1);
 69   oled.setCursor(0, 34);
 70   oled.print(amber ? "!! CLEARING !!" : "DIRECTION: GO");
 71
 72   oled.drawRect(0, 46, 128, 10, SSD1306_WHITE);
 73   int barWidth = map(secLeft, 0, totalSec, 0, 124);
 74   oled.fillRect(2, 48, barWidth, 6, SSD1306_WHITE);
 75
 76   oled.setTextSize(1);
 77   oled.setCursor(100, 56);
 78   oled.print(secLeft);
 79   oled.print("s");
 80
 81   oled.display();
 82 }
 83
 84 void runPhase(int pi) {
 85   int totalSec = phases[pi].durationMs / 1000;
 86
 87   allRed();
 88   for (int i = 0; i < 2; i++) setGreen(phases[pi].greenLanes[i]);
 89   for (int s = totalSec; s > 0; s--) {
 90     updateLCD(pi, s, false);
 91     updateOLED(pi, s, totalSec, false);
 92     delay(1000);
 93   }
 94
 95   for (int i = 0; i < 2; i++) setAmber(phases[pi].greenLanes[i]);
 96   int amberSec = AMBER_MS / 1000;
 97   for (int s = amberSec; s > 0; s--) {
 98     updateLCD(pi, s, true);
 99     updateOLED(pi, s, amberSec, true);
100     delay(1000);
101   }
102
103   allRed();
104   lcd.clear();
105   lcd.setCursor(0, 0); lcd.print("All red - safe  ");
106   lcd.setCursor(0, 1); lcd.print("Next phase...   ");
107   oled.clearDisplay();
108   oled.setTextSize(1);
```

```
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(20, 28); oled.print("-- ALL STOP --");
  oled.display();
  delay(ALL_RED_MS);
}

void setup() {
  delay(1000);

  Wire.begin();
  Wire.setClock(100000);
  delay(200);

  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  oled.clearDisplay();
  oled.setTextSize(1);
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(10, 20); oled.print("Smart Mini City");
  oled.setCursor(20, 36); oled.print("Traffic Ctrl");
  oled.display();

  delay(100);
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("Traffic System  ");
  lcd.setCursor(0, 1); lcd.print("Initializing... ");

  delay(200);

  for (int i = 0; i < 4; i++)
    for (int j = 0; j < 3; j++) {
      pinMode(lanes[i][j], OUTPUT);
      digitalWrite(lanes[i][j], LOW);
    }
  allRed();

  delay(2000);
}

void loop() {
  for (int i = 0; i < 2; i++) runPhase(i);
}
```

8 Solar Tracker

The solar tracker (firmware *SolarMind v2.0*) is a single-axis tracker that keeps a panel facing the brightest light. Two light-dependent resistors (LDRs) compare the light on the left and right, and a PID controller turns a servo until both sides receive equal light. At night the panel is parked in a flat position.

8.1 Function

- **AUTO mode** (green LED): the panel follows the light between 20° and 160°.
- **NIGHT mode** (red LED): when both LDR readings fall below 700, the panel is parked at 90° and the OLED shows *NIGHT MODE*.
- An OLED with four screens (selected with a push button) shows the LDR readings, angle and error; the PID gains and output; the current and target angles; and uptime.
- Readings are sent to the hub every 0.5 s.

8.2 Control method

The error signal is the difference between the two LDR readings, corrected by a fixed offset of 88 counts that compensates for the mismatch between the two sensors:

$$e = L - R - 88 \tag{3}$$

A positive error means the left side is brighter. The PID controller converts the error into a correction u, which is subtracted from the target servo angle:

$$u = K_{\mathrm{p}} e + K_{\mathrm{i}} \int e \, \mathrm{d}t + K_{\mathrm{d}} \, \mathrm{d}e/\mathrm{d}t \tag{4}$$

The proportional term ($K_{\mathrm{p}} = 0.15$) responds to the size of the error, the integral term ($K_{\mathrm{i}} = 0.002$) removes small steady errors, and the derivative term ($K_{\mathrm{d}} = 0.005$) damps the motion to prevent overshoot. The firmware adds five refinements:

1. **Filtering:** each LDR value is the average of the last three samples.
2. **Dead band:** errors smaller than 50 are treated as zero, so the panel does not hunt around the target.
3. **Adaptive gain:** K_{p} is multiplied by 1.5 for errors above 150 and by 2 above 300.
4. **Anti-windup:** the integral is limited to ±400, and the output to ±10 per step.
5. **Smoothing:** every 20 ms the servo moves 20 % of the remaining distance to its target.

8.3 Components

Table 17. Solar tracker components

Component	Qty	Purpose
Arduino UNO	1	Controller
LDR with voltage-divider resistor	2	Left and right light sensing
Servo motor	1	Panel rotation
Small solar panel	1	Tracked panel
0.96" OLED (SSD1306, I2C)	1	Display
LEDs (green, red)	2	Mode indication
Push button	1	Screen selection

8.4 Pin connections

Table 18. Solar tracker pin assignment

Signal	Arduino pin
Left LDR	A0
Right LDR	A1
Servo signal	D9
Green / red LED	D4 / D5
Push button	D3
OLED SDA / SCL (I2C, address 0x3C)	A4 / A5
Serial data to hub	D1 (TX) → GPIO 25

8.5 Software

Arduino IDE. Libraries: Wire, Servo, Adafruit GFX, Adafruit SSD1306.

8.6 Source code

Listing 6. Solar tracker firmware, SolarMind v2.0 (Arduino UNO)

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Servo.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);
Servo panelServo;

const int PIN_LDR_LEFT  = A0;
const int PIN_LDR_RIGHT = A1;
const int PIN_SERVO     = 9;
const int PIN_LED_GREEN = 4;
const int PIN_LED_RED   = 5;
const int PIN_BUTTON    = 3;

float Kp = 0.15, Ki = 0.002, Kd = 0.005;
float error = 0, prevError = 0, integral = 0, derivative = 0, pidOutput = 0;
float servoAngle = 90, servoTarget = 90;
```

```
const float SERVO_MIN = 20, SERVO_MAX = 160, LERP = 0.20;

const int SAMPLES = 3;
int leftBuf[SAMPLES] = {0};
int rightBuf[SAMPLES] = {0};
int bufIndex = 0;

const int DEAD_BAND = 50;
const float INTEGRAL_LIMIT = 400;
const int LOOP_MS = 20;
const int NIGHT_THRESHOLD =700;
const int LDR_OFFSET = 88;

enum Mode { AUTO, NIGHT };
Mode currentMode = AUTO;

int lastButtonState = HIGH;
unsigned long btnPressTime = 0;
bool longPressHandled = false;
const int LONG_PRESS_MS = 800;

int currentScreen = 0;
const int TOTAL_SCREENS = 4;

unsigned long lastTime = 0;
unsigned long startTime = 0;
unsigned long lastOledUpdate = 0;
float dt = 0;

void drawScreen0(int leftVal, int rightVal) {
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);  display.print("L:"); display.println(leftVal);
  display.setCursor(0, 10); display.print("R:"); display.println(rightVal);
  display.setCursor(0, 20); display.print("Angle:"); display.println((int)servoAngle);
  display.setCursor(0, 30); display.print("Err:"); display.println(error);
  display.setCursor(0, 40);
  if (currentMode == AUTO) display.print("AUTO");
  else                     display.print("NIGHT");
  display.display();
}

void drawScreen1() {
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);  display.print("Kp:"); display.println(Kp, 3);
  display.setCursor(0, 10); display.print("Ki:"); display.println(Ki, 3);
  display.setCursor(0, 20); display.print("Kd:"); display.println(Kd, 3);
  display.setCursor(0, 30); display.print("PID:"); display.println(pidOutput, 2);
  display.display();
}

void drawScreen2() {
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);  display.print("Angle:"); display.println((int)servoAngle);
  display.setCursor(0, 10); display.print("Target:"); display.println((int)servoTarget);
  display.setCursor(0, 20); display.print("Min:"); display.println((int)SERVO_MIN);
  display.setCursor(0, 30); display.print("Max:"); display.println((int)SERVO_MAX);
  display.display();
}

void drawScreen3() {
  unsigned long uptime = (millis() - startTime) / 1000;
  display.clearDisplay();
  display.setTextSize(1);
```

```
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);  display.print("Up:"); display.println(uptime);
  display.setCursor(0, 10); display.print("Loop:"); display.println(LOOP_MS);
  display.setCursor(0, 20); display.println("SolarMind v2.0");
  display.display();
}

void handleButton() {
  int btnState = digitalRead(PIN_BUTTON);
  if (btnState == LOW && lastButtonState == HIGH) {
    btnPressTime     = millis();
    longPressHandled = false;
  }
  if (btnState == LOW && !longPressHandled) {
    if (millis() - btnPressTime > LONG_PRESS_MS) {
      longPressHandled = true;
    }
  }
  if (btnState == HIGH && lastButtonState == LOW) {
    if (!longPressHandled)
      currentScreen = (currentScreen + 1) % TOTAL_SCREENS;
  }
  lastButtonState = btnState;
}

void setup() {
  Serial.begin(9600);
  delay(3000);
  Wire.begin();
  delay(1000);

  bool oledOK = false;
  for (int i = 0; i < 10; i++) {
    if (display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
      oledOK = true;
      Serial.println("OLED OK!");
      break;
    }
    Serial.println("Retrying...");
    delay(500);
  }
  if (!oledOK) { Serial.println("FAILED!"); while(true); }

  display.clearDisplay();
  display.setTextSize(2);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(18, 8);  display.println("SOLAR");
  display.setCursor(18, 30); display.println("MIND");
  display.setTextSize(1);
  display.setCursor(28, 54); display.println("v2.0 booting...");
  display.display();

  pinMode(PIN_LED_GREEN, OUTPUT);
  pinMode(PIN_LED_RED,   OUTPUT);
  pinMode(PIN_BUTTON,    INPUT);

  for (int i = 0; i < 3; i++) {
    digitalWrite(PIN_LED_GREEN, HIGH); delay(200);
    digitalWrite(PIN_LED_GREEN, LOW);
    digitalWrite(PIN_LED_RED,   HIGH); delay(200);
    digitalWrite(PIN_LED_RED,   LOW);
  }

  panelServo.attach(PIN_SERVO);
  panelServo.write(90);
  delay(500);

  startTime = millis();
  lastTime  = millis();
```

```
  Serial.println("=== SolarMind v2.0 Ready ===");
}

void loop() {
  unsigned long now = millis();
  dt = (now - lastTime) / 1000.0;
  lastTime = now;
  if (dt < 0.001) return;

  handleButton();

  leftBuf[bufIndex]  = analogRead(PIN_LDR_LEFT);
  rightBuf[bufIndex] = analogRead(PIN_LDR_RIGHT);
  bufIndex = (bufIndex + 1) % SAMPLES;

  int leftVal = 0, rightVal = 0;
  for (int i = 0; i < SAMPLES; i++) {
    leftVal  += leftBuf[i];
    rightVal += rightBuf[i];
  }
  leftVal  /= SAMPLES;
  rightVal /= SAMPLES;

  if (leftVal < NIGHT_THRESHOLD && rightVal < NIGHT_THRESHOLD) {
    currentMode = NIGHT;
  } else if (currentMode == NIGHT) {
    currentMode = AUTO;
  }

  if (currentMode == AUTO) {
    digitalWrite(PIN_LED_GREEN, HIGH);
    digitalWrite(PIN_LED_RED,   LOW);

    error = (float)(leftVal - rightVal - LDR_OFFSET);
    if (abs(error) < DEAD_BAND) {
      error    = 0;
      integral = 0;
      servoTarget += (90 - servoTarget) * 0.05;
    }
    float adaptKp = Kp;
    if (abs(error) > 300)      adaptKp = Kp * 2.0;
    else if (abs(error) > 150) adaptKp = Kp * 1.5;

    integral  += error * dt;
    integral   = constrain(integral, -INTEGRAL_LIMIT, INTEGRAL_LIMIT);
    derivative = (error - prevError) / dt;
    pidOutput  = (adaptKp * error) + (Ki * integral) + (Kd * derivative);
    prevError  = error;

    if (abs(pidOutput) < 0.5) pidOutput = 0;
    pidOutput   = constrain(pidOutput, -10, 10);
    servoTarget -= pidOutput;
    servoTarget  = constrain(servoTarget, SERVO_MIN, SERVO_MAX);

  } else if (currentMode == NIGHT) {
    servoTarget = 90;
    integral    = 0;
    digitalWrite(PIN_LED_GREEN, LOW);
    digitalWrite(PIN_LED_RED,   HIGH);
  }

  servoAngle += (servoTarget - servoAngle) * LERP;
  panelServo.write((int)servoAngle);

  if (currentMode == NIGHT) {
    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(10, 10); display.println("NIGHT");
```

```
    display.setCursor(10, 30); display.println("MODE");
    display.setTextSize(1);
    display.setCursor(10, 54); display.println("Parked at 90 deg");
    display.display();
  } else if (millis() - lastOledUpdate > 200) {
    lastOledUpdate = millis();
    switch (currentScreen) {
      case 0: drawScreen0(leftVal, rightVal); break;
      case 1: drawScreen1(); break;
      case 2: drawScreen2(); break;
      case 3: drawScreen3(); break;
    }
  }

  static unsigned long lastPrint = 0;
  if (millis() - lastPrint > 500) {
    lastPrint = millis();
    Serial.print("L:"); Serial.print(leftVal);
    Serial.print(",R:"); Serial.print(rightVal);
    Serial.print(",Err:"); Serial.print(error);
    Serial.print(",Ang:"); Serial.println(servoAngle);
  }

  delay(LOOP_MS);
}
```

9 Weather Station and Central Hub

The weather station is built on an ESP32. It measures temperature, humidity and light, works out how the air actually feels to a person, and displays the results on a 16×2 LCD. Because it has Wi-Fi and several serial ports, the same ESP32 also serves as the hub of the city.

9.1 Function

- Reads temperature and humidity from a DHT22 sensor every 3 s, and light level from an LDR module.
- Calculates the **heat index** (apparent temperature), **dew point**, absolute humidity and vapour pressure.
- Labels the weather (e.g. Pleasant, Warm, Humid, Tropical Heat) and rates comfort from the heat index (Cool, Comfortable, Caution, Extreme Caution, Danger).
- Shows six LCD screens, selected with a push button: temperature/humidity; heat index/dew point; weather label and comfort; light level and day/night; gas status and pump; door status and last flat.
- Sounds a continuous alarm and shows *FIRE ALERT – EVACUATE NOW* when the gas monitor reports danger, and a single beep with a 3-second warning screen when the heat index reaches 40 °C or humidity reaches 90 %. Alarms are suppressed for the first 8 s after power-on.

9.2 Hub operation

1. **Collecting.** Four Arduinos send one-line serial messages to the ESP32 (see Section 2.1). The hub reads each line and extracts the values by locating the field names in the text.
2. **Receiving the door.** The RFID door's ESP32 posts its data over Wi-Fi to the hub's built-in web server (`/sensordata`). A second address, `/data`, returns all current readings as JSON.
3. **Uploading.** Every 8 s the hub combines all readings from the seven systems into one JSON object and sends it over HTTPS to the Cloudflare Worker.
4. **Status monitoring.** A system that has not reported for 10–30 s is marked offline.
5. **Reliability.** The hub tries up to four saved Wi-Fi networks in turn, synchronises its clock with an NTP time server (UTC+6), stores a 2-hour history (24 readings at 5-minute intervals), runs a 30-second watchdog timer, and announces itself on the local network as `orbis.local`.

9.3 Components

Table 19. Weather station components

Component	Qty	Purpose
ESP32 development board	1	Controller with Wi-Fi
DHT22 temperature and humidity sensor	1	Weather measurement
LDR module (analog and digital outputs)	1	Light level, day/night
16×2 LCD with I2C adapter	1	Display
Buzzer	1	Alarms
Push button	1	Screen selection

9.4 Pin connections

Table 20. Weather station and hub pin assignment

Signal	ESP32 pin
DHT22 data	GPIO 4
LCD SDA / SCL (I2C, address 0x27)	GPIO 21 / GPIO 22
LDR analog output	GPIO 34
LDR digital output	GPIO 35
Buzzer	GPIO 32
Push button (internal pull-up)	GPIO 33
Smart dustbin TX (Serial1 RX)	GPIO 13
Smart parking TX (Serial2 RX)	GPIO 16
Traffic lights TX (software serial)	GPIO 14
Solar tracker TX (software serial)	GPIO 25

9.5 Software

Arduino IDE with the ESP32 board package. Libraries: WiFi, WebServer, HTTPClient, WiFiClientSecure and ESPmDNS (ESP32 core); DHT sensor library; LiquidCrystal_I2C; ArduinoJson; SoftwareSerial.

9.6 Source code

Wi-Fi network names, passwords and the cloud address are replaced with placeholders.

Listing 7. Weather station and hub firmware (ESP32)

```
#include <WiFi.h>
#include <WebServer.h>
#include <DHT.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <ArduinoJson.h>
#include <time.h>
#include <math.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
#include "esp_task_wdt.h"
#include <ESPmDNS.h>
#include <SoftwareSerial.h>
SoftwareSerial trafficSerial(14, -1);
SoftwareSerial solarSerial(25, -1);

struct WifiCred { const char* ssid; const char* pass; };
WifiCred networks[] = {
  { "WIFI_NAME_1", "PASSWORD_1" },
  { "WIFI_NAME_2", "PASSWORD_2" },
  { "WIFI_NAME_3", "PASSWORD_3" },
  { "WIFI_NAME_4", "PASSWORD_4" },
};
const int NET_COUNT = sizeof(networks) / sizeof(networks[0]);
const char* LOCATION      = "Chittagong, BD";

#define DHT_PIN   4
#define DHT_TYPE  DHT22
#define LCD_ADDR  0x27
```

```
#define LCD_COLS  16
#define LCD_ROWS  2
#define LDR_A     34
#define LDR_D     35
#define BUZZER    32
#define BUTTON    33

DHT dht(DHT_PIN, DHT_TYPE);
LiquidCrystal_I2C lcd(LCD_ADDR, LCD_COLS, LCD_ROWS);
WebServer server(80);

unsigned long fireAlertShownAt = 0;
bool fireAlertActive = false;
unsigned long heatAlertShownAt = 0;
bool heatAlertActive = false;
int heatBeepCount = 0;
bool heatBeepDone = false;
bool lastHeatBeepState = false;

unsigned long lastLCDUpdate = 0;
#define LCD_INT 1000UL

float temperature=0, humidity=0, heatIndex=0;
int   lightLevel=0;
bool  isDay=false, sensorOk=false, buzzerOn=false;

bool bootComplete = false;
unsigned long bootTime = 0;

String doorStatus  = "LOCKED";
String lastUID     = "None";
String lastFlat    = "None";
int    mq2Val      = 0;
int    mq135Val    = 0;
String gasStatus   = "SAFE";
String pumpStatus  = "OFF";
unsigned long lastSensorData = 0;

int parkingSlots = 4;
String parkingStatus = "OPEN";
unsigned long lastParkingData = 0;

int trashFill = 0;
int trashDepth = 0;
String trashLid = "CLOSED";
String trashStatus = "OK";
unsigned long lastTrashData = 0;

int trafficPhase = 0;
String trafficDir = "NONE";
int trafficTime = 0;
String trafficState = "RED";
int trafficTotal = 0;
unsigned long lastTrafficData = 0;

int solarLeft = 0;
int solarRight = 0;
float solarAngle = 90;
float solarError = 0;
String solarMode = "AUTO";
unsigned long lastSolarData = 0;

#define HIST 24
float tHist[HIST], hHist[HIST]; long tsHist[HIST];
int hIdx=0, hCount=0;

unsigned long lastSensor=0, lastHistory=0, lastBuzzer=0, lastDebounce=0;
int lcdScreen=0; bool lastButtonState=HIGH;

```

```
#define SENSOR_INT   3000UL
#define HIST_INT     300000UL
#define BUZZER_INT   500UL
#define DEBOUNCE     200UL
#define HI_DANGER    40.0
#define HUM_DANGER   90.0
#define TOTAL_SCREENS 6

uint8_t degChar[8]={0b00110,0b01001,0b01001,0b00110,0b00000,0b00000,0b00000,0b00000};

float dewPoint(float t,float h){float a=17.27f,b=237.7f,al=((a*t)/(b+t))+log(h/100.0f);return (b*al)/(a-al);}
float absHumidity(float t,float h){float es=6.1078f*expf((17.27f*t)/(t+237.3f));return (216.7f*(h/100.0f*es))/(273.15f+t);}
float vaporPressure(float t,float h){return h/100.0f*6.1078f*expf((17.27f*t)/(t+237.3f));}
const char* condition(float t,float h){
  if(h>85&&t>30)return "Tropical Heat";if(h>80)return "Humid";if(t>38)return "Extreme Heat";
  if(t>33)return "Hot";if(t>28)return "Warm";if(t>20)return "Pleasant";if(t>15)return "Cool";
  if(t>8)return "Cold";return "Freezing";
}
const char* comfort(float hi){
  if(hi>=54)return "DANGER";if(hi>=41)return "EXT CAUTION";if(hi>=33)return "CAUTION";
  if(hi>=27)return "COMFORTABLE";return "COOL";
}

 void handleBuzzer(){
  if(!bootComplete){
    if(millis()-bootTime < 8000){ return; }
    bootComplete = true;
  }

  bool fireNow = (gasStatus == "DANGER");
  bool heatNow = (heatIndex >= HI_DANGER || humidity >= HUM_DANGER);

  if(fireNow){
    // Reset heat beep state
    heatBeepCount = 0;
    heatBeepDone = false;
    lastHeatBeepState = false;
    // Continuous beep for fire
    unsigned long n = millis();
    if(n - lastBuzzer >= BUZZER_INT){
      lastBuzzer = n;
      buzzerOn = !buzzerOn;
      digitalWrite(BUZZER, buzzerOn ? HIGH : LOW);
    }
    return;
  }

  if(heatNow){
    if(!lastHeatBeepState){
      heatBeepCount = 0;
      heatBeepDone = false;
      lastHeatBeepState = true;
    }
    if(!heatBeepDone){
      unsigned long n = millis();
      if(n - lastBuzzer >= BUZZER_INT){
        lastBuzzer = n;
        if(buzzerOn){
          heatBeepCount++;
          if(heatBeepCount >= 1) heatBeepDone = true;
        }
        buzzerOn = !buzzerOn;
        digitalWrite(BUZZER, buzzerOn ? HIGH : LOW);
      }
    } else {
      digitalWrite(BUZZER, LOW);
      buzzerOn = false;
```

```
    }
    return;
  }

  // Safe — reset all
  digitalWrite(BUZZER, LOW);
  buzzerOn = false;
  heatBeepCount = 0;
  heatBeepDone = false;
  lastHeatBeepState = false;
}

void handleButton(){
  bool r=digitalRead(BUTTON); unsigned long n=millis();
  if(r==LOW&&lastButtonState==HIGH&&(n-lastDebounce)>DEBOUNCE){
    lastDebounce=n; lcdScreen=(lcdScreen+1)%TOTAL_SCREENS; updateLCD();
  }
  lastButtonState=r;
}

void updateLCD(){
  if(!sensorOk){
    lcd.clear();
    lcd.setCursor(0,0); lcd.print("Sensor error!");
    lcd.setCursor(0,1); lcd.print("Check DHT22");
    return;
  }

  // FIRE ALERT — highest priority, 5 seconds
  if(gasStatus=="DANGER"){
    if(!fireAlertActive){
      fireAlertActive=true;
      fireAlertShownAt=millis();
      lcd.clear();
      lcd.setCursor(0,0); lcd.print("!! FIRE ALERT !!");
      lcd.setCursor(0,1); lcd.print("EVACUATE NOW!!! ");
      return;
    } else if(millis()-fireAlertShownAt < 5000){
      return;
    }
    fireAlertActive=false;
  } else {
    fireAlertActive=false;
  }

  // HEAT/HUMIDITY WARNING — show once for 3 seconds then return to normal
  bool hiHigh = heatIndex>=HI_DANGER;
  bool humHigh = humidity>=HUM_DANGER;

  // If currently showing alert, check if 3 seconds passed
  if(heatAlertActive){
    if(millis()-heatAlertShownAt < 3000){
      return; // still within 3 seconds, keep showing
    }
    heatAlertActive=false; // 3 seconds done, go to normal screen
  }
  // Only trigger NEW alert if condition is true and beep just started
  else if((hiHigh || humHigh) && !heatAlertActive && !heatBeepDone){
    heatAlertActive=true;
    heatAlertShownAt=millis();
    lcd.clear();
    char buf[8];
    if(hiHigh && humHigh){
      lcd.setCursor(0,0); lcd.print("!!HEAT WARNING!!");
      lcd.setCursor(0,1);
      lcd.print("HI:"); dtostrf(heatIndex,4,1,buf); lcd.print(buf);
      lcd.print(" H:"); dtostrf(humidity,3,0,buf); lcd.print(buf); lcd.print("%");
    } else if(hiHigh){
      lcd.setCursor(0,0); lcd.print("!! HIGH HI !!");
```

```
      lcd.setCursor(0,1);
      lcd.print("HeatIdx:"); dtostrf(heatIndex,5,1,buf); lcd.print(buf);
    } else {
      lcd.setCursor(0,0); lcd.print("!! HIGH HUMID !!");
      lcd.setCursor(0,1);
      lcd.print("Humidity:"); dtostrf(humidity,4,1,buf); lcd.print(buf); lcd.print("%");
    }
    return;
  }

  // NORMAL SCREENS
  char buf[8]; lcd.clear();
  lcd.setCursor(13,1); lcd.print(lcdScreen+1); lcd.print("/"); lcd.print(TOTAL_SCREENS);
  switch(lcdScreen){
    case 0:
lcd.setCursor(0,0);lcd.print("Temp:");dtostrf(temperature,5,1,buf);lcd.print(buf);lcd.write(0);lcd.print("C");
lcd.setCursor(0,1);lcd.print("Hum:");dtostrf(humidity,4,1,buf);lcd.print(buf);lcd.print("%");
      break;
    case 1:
lcd.setCursor(0,0);lcd.print("HI:");dtostrf(heatIndex,5,1,buf);lcd.print(buf);lcd.write(0);lcd.print("C");
lcd.setCursor(0,1);lcd.print("Dew:");dtostrf(dewPoint(temperature,humidity),4,1,buf);lcd.print(buf);lcd.write(0);lcd.print("C");
      break;
    case 2:
      lcd.setCursor(0,0);lcd.print(condition(temperature,humidity));
      lcd.setCursor(0,1);lcd.print(comfort(heatIndex));
      break;
    case 3:
      lcd.setCursor(0,0);lcd.print("Light:");lcd.print(lightLevel);lcd.print("%");
      lcd.setCursor(0,1);lcd.print(isDay?"Daytime":"Nighttime");
      break;
    case 4:
      lcd.setCursor(0,0);lcd.print("Gas:");lcd.print(gasStatus);
      lcd.setCursor(0,1);lcd.print("MQ2:");lcd.print(mq2Val);lcd.print("P:");lcd.print(pumpStatus);
      break;
    case 5:
      lcd.setCursor(0,0);lcd.print("Door:");lcd.print(doorStatus);
      lcd.setCursor(0,1);lcd.print(lastFlat);
      break;
  }
}

void handleSensorData(){
  if(server.hasArg("door"))  doorStatus = server.arg("door");
  if(server.hasArg("uid"))   lastUID    = server.arg("uid");
  if(server.hasArg("flat"))  lastFlat   = server.arg("flat");
  if(server.hasArg("mq2"))   mq2Val     = server.arg("mq2").toInt();
  if(server.hasArg("mq135")) mq135Val   = server.arg("mq135").toInt();
  if(server.hasArg("gas"))   gasStatus  = server.arg("gas");
  if(server.hasArg("pump"))  pumpStatus = server.arg("pump");
  lastSensorData = millis();
  server.send(200,"text/plain","OK");
}

void handleData(){
  StaticJsonDocument<2048> doc;
  doc["temperature"]   = round(temperature*10)/10.0;
  doc["humidity"]      = round(humidity*10)/10.0;
  doc["heatIndex"]     = round(heatIndex*10)/10.0;
  doc["dewPoint"]      = round(dewPoint(temperature,humidity)*10)/10.0;
  doc["absHumidity"]   = absHumidity(temperature,humidity);
  doc["vaporPressure"] = vaporPressure(temperature,humidity);
```

```
  doc["condition"]     = condition(temperature,humidity);
  doc["comfort"]       = comfort(heatIndex);
  doc["sensorOk"]      = sensorOk;
  doc["location"]      = LOCATION;
  doc["lightLevel"]    = lightLevel;
  doc["isDay"]         = isDay;
  doc["alert"]         = (gasStatus=="DANGER");
  doc["warnHeat"]      = (heatIndex>=HI_DANGER);
  doc["warnHumidity"]  = (humidity>=HUM_DANGER);
  doc["door"]          = doorStatus;
  doc["uid"]           = lastUID;
  doc["flat"]          = lastFlat;
  doc["mq2"]           = mq2Val;
  doc["mq135"]         = mq135Val;
  doc["gas"]           = gasStatus;
  doc["pump"]          = pumpStatus;
  doc["rfidOnline"]    = (millis()-lastSensorData < 10000);
  doc["parking_slots"]  = parkingSlots;
  doc["parking_status"] = parkingStatus;
  doc["parking_online"] = (millis()-lastParkingData < 30000);
  doc["trash_fill"]   = trashFill;
  doc["trash_depth"]  = trashDepth;
  doc["trash_lid"]    = trashLid;
  doc["trash_status"] = trashStatus;
  doc["trash_online"] = (millis()-lastTrashData < 15000);

  JsonObject hist=doc.createNestedObject("history");
  JsonArray ta=hist.createNestedArray("times");
  JsonArray tp=hist.createNestedArray("temps");
  JsonArray hp=hist.createNestedArray("hums");
  int start=(hCount<HIST)?0:hIdx; int cnt=min(hCount,HIST);
  for(int i=0;i<cnt;i++){int idx=
(start+i)%HIST;ta.add(tsHist[idx]);tp.add(round(tHist[idx]*10)/10.0);hp.add(round(hHist[idx]*10)/10
.0);}
  String out; serializeJson(doc,out);
  server.sendHeader("Access-Control-Allow-Origin","*");
  server.send(200,"application/json",out);
}

unsigned long lastSupabasePost = 0;
#define SUPABASE_INT 8000UL

void postToSupabase() {
  if (WiFi.status() != WL_CONNECTED) return;
  WiFiClientSecure client;
  client.setInsecure();
  HTTPClient http;
  http.begin(client, "https://YOUR-WORKER.workers.dev");
  http.setTimeout(4000);
  http.addHeader("Content-Type", "application/json");

  StaticJsonDocument<3072> doc;
  doc["temperature"]   = round(temperature*10)/10.0;
  doc["humidity"]      = round(humidity*10)/10.0;
  doc["heat_index"]    = round(heatIndex*10)/10.0;
  doc["dew_point"]     = round(dewPoint(temperature,humidity)*10)/10.0;
  doc["abs_humidity"]  = absHumidity(temperature,humidity);
  doc["vapor_pressure"]= vaporPressure(temperature,humidity);
  doc["condition"]     = condition(temperature,humidity);
  doc["comfort"]       = comfort(heatIndex);
  doc["sensor_ok"]     = sensorOk;
  doc["light_level"]   = lightLevel;
  doc["is_day"]        = isDay;
  doc["alert"]         = (gasStatus=="DANGER");
  doc["warnHeat"]     = (heatIndex>=HI_DANGER);
  doc["warnHumidity"] = (humidity>=HUM_DANGER);
  doc["door"]          = doorStatus;
  doc["uid"]           = lastUID;
  doc["flat"]          = lastFlat;
```

```
  doc["mq2"]           = mq2Val;
  doc["mq135"]         = mq135Val;
  doc["gas"]           = gasStatus;
  doc["pump"]          = pumpStatus;
  doc["rfid_online"]   = (millis()-lastSensorData < 10000);
  doc["parking_slots"]  = parkingSlots;
  doc["parking_status"] = parkingStatus;
  doc["parking_online"] = (millis()-lastParkingData < 30000);
  doc["trash_fill"]   = trashFill;
  doc["trash_depth"]  = trashDepth;
  doc["trash_lid"]    = trashLid;
  doc["trash_status"] = trashStatus;
  doc["trash_online"] = (millis()-lastTrashData < 15000);
  doc["traffic_phase"]  = trafficPhase;
  doc["traffic_dir"]    = trafficDir;
  doc["traffic_time"]   = trafficTime;
  doc["traffic_state"]  = trafficState;
  doc["traffic_total"]  = trafficTotal;
  doc["traffic_online"] = (millis()-lastTrafficData < 15000);
  doc["solar_left"]   = solarLeft;
  doc["solar_right"]  = solarRight;
  doc["solar_angle"]  = solarAngle;
  doc["solar_error"]  = solarError;
  doc["solar_mode"]   = solarMode;
  doc["solar_online"] = (millis()-lastSolarData < 15000);

  String body;
  serializeJson(doc, body);
  int httpCode = http.POST(body);
  Serial.println("Supabase: " + String(httpCode));
  http.end();
  client.stop();
  yield();
}

void setup(){
  Serial.begin(115200);
  Serial1.begin(9600, SERIAL_8N1, 13, -1);
  Serial2.begin(9600, SERIAL_8N1, 16, -1);
  trafficSerial.begin(9600);
  solarSerial.begin(9600);
  MDNS.begin("orbis");
  esp_task_wdt_config_t wdt_config = {
  .timeout_ms = 30000,
  .idle_core_mask = 0,
  .trigger_panic = false
};
esp_task_wdt_init(&wdt_config);
  pinMode(LDR_D,INPUT); pinMode(BUZZER,OUTPUT); pinMode(BUTTON,INPUT_PULLUP);
  digitalWrite(BUZZER,LOW);
  Wire.begin(21,22);
  lcd.init(); lcd.backlight(); lcd.createChar(0,degChar);
  lcd.clear(); lcd.setCursor(0,0); lcd.print("  ORBIS v1.0");
  lcd.setCursor(0,1); lcd.print("  Starting...");
  delay(1500);
  dht.begin(); delay(2000);
  float t=dht.readTemperature(); float hh=dht.readHumidity();
  sensorOk=(!isnan(t)&&!isnan(hh));
  if(sensorOk){temperature=t;humidity=hh;heatIndex=dht.computeHeatIndex(t,hh,false);}
  lightLevel=map(analogRead(LDR_A),4095,0,0,100);
  isDay=digitalRead(LDR_D)==LOW;
  lcd.clear(); lcd.setCursor(0,0); lcd.print("Connecting WiFi");
  WiFi.mode(WIFI_STA);
  WiFi.disconnect(true);
  delay(500);
  for(int i=0;i<NET_COUNT;i++){
    Serial.print("Trying: ");Serial.println(networks[i].ssid);
    WiFi.begin(networks[i].ssid,networks[i].pass);
    int tries=0;
```

```
    while(WiFi.status()!=WL_CONNECTED&&tries<20){delay(500);tries++;}
    if(WiFi.status()==WL_CONNECTED){
      Serial.println("Connected: "+String(networks[i].ssid));
      break;
    }
    WiFi.disconnect(true);
    delay(300);
  }
  if(WiFi.status()==WL_CONNECTED){
    lcd.clear(); lcd.setCursor(0,0); lcd.print(WiFi.localIP());
    lcd.setCursor(0,1); lcd.print("ORBIS Ready!");
    Serial.print("IP: "); Serial.println(WiFi.localIP());
    delay(3000);
  } else {
    lcd.clear(); lcd.setCursor(0,0); lcd.print("WiFi failed!"); delay(2000);
  }
  configTime(6*3600,0,"pool.ntp.org");
  server.on("/data",handleData);
  server.on("/sensordata",HTTP_POST,handleSensorData);
  server.begin();
  updateLCD();
  postToSupabase(); // immediate first post on boot
  bootTime = millis(); // start boot silence timer
}

void loop(){
  server.handleClient();
  if (Serial2.available()) {
    String line = Serial2.readStringUntil('\n');
    line.trim();
    int i1 = line.indexOf("SLOTS=");
    int i2 = line.indexOf(",STATUS=");
    if (i1 >= 0 && i2 >= 0) {
      parkingSlots = line.substring(i1 + 6, i2).toInt();
      parkingStatus = line.substring(i2 + 8);
      lastParkingData = millis();
      Serial.println("Parking: slots=" + String(parkingSlots) + " status=" + parkingStatus);
    }
  }

  if (Serial1.available()) {
    String line = Serial1.readStringUntil('\n');
    line.trim();
    if (line.startsWith("FILL=")) {
      int i1 = line.indexOf("FILL=");
      int i2 = line.indexOf(",DEPTH=");
      int i3 = line.indexOf(",LID=");
      int i4 = line.indexOf(",STATUS=");
      if (i1 >= 0 && i2 >= 0 && i3 >= 0 && i4 >= 0) {
        trashFill   = line.substring(i1 + 5, i2).toInt();
        trashDepth  = line.substring(i2 + 7, i3).toInt();
        trashLid    = line.substring(i3 + 5, i4);
        trashStatus = line.substring(i4 + 8);
        lastTrashData = millis();
        Serial.println("Trash: fill=" + String(trashFill) + "% depth=" + String(trashDepth) +
"cm");
      }
    }
  }

    if (trafficSerial.available()) {
    String line = trafficSerial.readStringUntil('\n');
    line.trim();
    if (line.startsWith("PHASE=")) {
      int i1 = line.indexOf("PHASE=");
      int i2 = line.indexOf(",DIR=");
      int i3 = line.indexOf(",TIME=");
      int i4 = line.indexOf(",STATE=");
      int i5 = line.indexOf(",TOTAL=");
```

```
      if (i1 >= 0 && i2 >= 0 && i3 >= 0 && i4 >= 0 && i5 >= 0) {
        trafficPhase = line.substring(i1 + 6, i2).toInt();
        trafficDir   = line.substring(i2 + 5, i3);
        trafficTime  = line.substring(i3 + 6, i4).toInt();
        trafficState = line.substring(i4 + 7, i5);
        trafficTotal = line.substring(i5 + 7).toInt();
        lastTrafficData = millis();
        Serial.println("Traffic: phase=" + String(trafficPhase) + " dir=" + trafficDir + "
state=" + trafficState);
      }
    }
  }

  if (solarSerial.available()) {
    String line = solarSerial.readStringUntil('\n');
    line.trim();
    if (line.startsWith("L:")) {
      int i1 = line.indexOf("L:");
      int i2 = line.indexOf(",R:");
      int i3 = line.indexOf(",Err:");
      int i4 = line.indexOf(",Ang:");
      if (i1 >= 0 && i2 >= 0 && i3 >= 0 && i4 >= 0) {
        solarLeft  = line.substring(i1 + 2, i2).toInt();
        solarRight = line.substring(i2 + 3, i3).toInt();
        solarError = line.substring(i3 + 5, i4).toFloat();
        solarAngle = line.substring(i4 + 5).toFloat();
        solarMode  = (solarLeft < 700 && solarRight < 700) ? "NIGHT" : "AUTO";
        lastSolarData = millis();
        Serial.println("Solar: angle=" + String(solarAngle) + " mode=" + solarMode);
      }
    }
  }

  unsigned long now=millis();
  if(now-lastSensor>=SENSOR_INT){
    lastSensor=now;
    float t=dht.readTemperature(); float hh=dht.readHumidity();
    sensorOk=(!isnan(t)&&!isnan(hh));
    if(sensorOk){temperature=t;humidity=hh;heatIndex=dht.computeHeatIndex(t,hh,false);}
    lightLevel=map(analogRead(LDR_A),4095,0,0,100);
    isDay=digitalRead(LDR_D)==HIGH;
  }
  handleBuzzer();
  handleButton();
  if(now-lastLCDUpdate>=LCD_INT){
    lastLCDUpdate=now;
    updateLCD();
  }
  if(now-lastSupabasePost>=SUPABASE_INT){
    lastSupabasePost=now;
    postToSupabase();
  }
  if(now-lastHistory>=HIST_INT){
    lastHistory=now; time_t epoch; time(&epoch);
    tHist[hIdx]=temperature; hHist[hIdx]=humidity; tsHist[hIdx]=(long)epoch;
    hIdx=(hIdx+1)%HIST; if(hCount<HIST)hCount++;
  }
}
```

10 Cloud Relay, Database and Dashboard

Readings travel from the hub to a cloud database and are displayed on a public website. Three free services are used: a Cloudflare Worker as a relay, a Supabase database for storage and Vercel to host the dashboard.

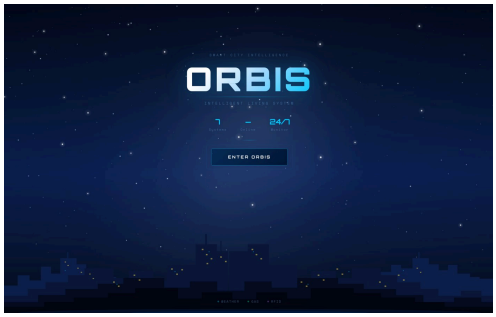


Figure 6. Landing page of the ORBIS web dashboard.

10.1 Data path

1. Every 8 s the hub sends one JSON object containing every reading to the Cloudflare Worker over HTTPS.
2. The Worker inserts it as a new row in the Supabase table `sensor_data`. Every message is kept, so the database holds the complete history.
3. The dashboard requests the newest row from the same Worker and updates the page.

Placing the Worker between the devices and the database keeps the database key on Cloudflare's server; it never appears in the ESP32 firmware or in the website's code.

10.2 Cloudflare Worker

A Cloudflare Worker is a short JavaScript program that runs on Cloudflare's servers whenever its web address is requested. The ORBIS Worker handles three request types.

Table 21. Requests handled by the Worker

Method	Sent by	Action
POST	Hub ESP32	Inserts the JSON body as a new row in Supabase
GET	Dashboard	Returns the newest row
OPTIONS	Web browser	Permits cross-origin requests (CORS)

The database address and key are replaced with placeholders.

Listing 8. Cloudflare Worker (JavaScript)

```
export default {
  async fetch(request, env) {
    if (request.method === "OPTIONS") {
      return new Response(null, {
        headers: {
          "Access-Control-Allow-Origin": "*",
          "Access-Control-Allow-Methods": "POST, GET",
          "Access-Control-Allow-Headers": "Content-Type",
        },
      });
    }

    if (request.method === "POST") {
      const data = await request.json();
      const res = await fetch(
        "https://YOUR-PROJECT.supabase.co/rest/v1/sensor_data",
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json",
```

```
            "apikey": SUPABASE_KEY,            // key hidden
            "Authorization": "Bearer " + SUPABASE_KEY,
            "Prefer": "return=minimal",
          },
          body: JSON.stringify(data),
        }
      );
      return new Response(String(res.status), {
        headers: { "Access-Control-Allow-Origin": "*" },
      });
    }

    if (request.method === "GET") {
      const res = await fetch(
        "https://YOUR-PROJECT.supabase.co/rest/v1/sensor_data?
select=*&order=created_at.desc&limit=1",
        {
          headers: {
            "apikey": SUPABASE_KEY,
            "Authorization": "Bearer " + SUPABASE_KEY,
          },
        }
      );
      const data = await res.text();
      return new Response(data, {
        headers: { "Content-Type": "application/json", "Access-Control-Allow-Origin": "*" },
      });
    }

    return new Response("ORBIS Relay OK", {
      headers: { "Access-Control-Allow-Origin": "*" },
    });
  },
};
```

10.3 Supabase database

Supabase is a hosted database service based on PostgreSQL. The project uses one table, `sensor_data`, with one column for each value sent by the hub (weather, gas, door, parking, dustbin, traffic, solar and an online flag for each system) and a `created_at` timestamp.

10.4 Web dashboard

The dashboard is a single-page website written in HTML, CSS and JavaScript and hosted on Vercel. It was built by Afrin Mostafa Riya.

- Landing page with an animated city skyline and the number of systems online.
- Systems page listing each system with its live status.
- Weather view with temperature, a comfort scale and a temperature/humidity chart.
- Fire alert: the page flashes red, a banner appears and a pop-up warns the viewer.
- Responsive layout for phones and computers.

Live: orbis-dashboard-amber.vercel.app

10.5 Tools used in the project

Table 22. Software and services

Tool	Used for
Arduino IDE	Writing and uploading firmware to all seven boards
Cloudflare Workers	HTTPS relay between the hub and the database
Supabase (PostgreSQL)	Storage of every reading
Vercel	Hosting the dashboard
HTML, CSS, JavaScript	Dashboard

11 References

Published work on the same kinds of systems as ORBIS, for further reading.

1. H. A. S. Azmira and T. Kian Sek, "IoT-Based Weather Monitoring Station for Smart City," *Evolution in Electrical and Electronic Engineering*, vol. 3, no. 2, pp. 561–568, 2022. periodical.uthm.edu.my/index.php/eeee/article/view/7776

2. M. A. Hannan, A. S. Mohd Zain, F. Salehuddin, H. Hazura, S. K. Idris, A. R. Hanim *et al.*, "Development of LPG Leakage Detector System using Arduino with Internet of Things (IoT)," *Journal of Telecommunication, Electronic and Computer Engineering*, vol. 10, no. 2-7, pp. 91–95. jtec.utem.edu.my/jtec/article/view/4432

3. R. Suhail, "Automated door access based on RFID using Arduino," in *Proc. 2nd Int. Conf. on ICT for Digital, Smart, and Sustainable Development (ICIDSSD)*, EAI, 2021. doi: 10.4108/eai.27-2-2020.2303113

4. M. M. Abd Kadir, M. N. Osman, N. A. Othman and K. A. Sedek, "IoT based Car Parking Management System using IR Sensor," 2020. ir.uitm.edu.my/id/eprint/43518

5. S. Abba and C. I. Light, "IoT-Based Framework for Smart Waste Monitoring and Control System: A Case Study of Smart Cities," in *Proc. 7th Int. Electronic Conf. on Sensors and Applications*, MDPI, 2020. doi: 10.3390/ecsa-7-08224

6. A. I. bin Ibrahim, F. D. binti Abdul Rahman and M. bin Rohaizat, "Dual Axes Solar Tracker," *International Journal of Electrical and Computer Engineering (IJECE)*, vol. 8, no. 3, pp. 1887–1892, 2018. doi: 10.11591/ijece.v8i3.pp1887-1892

7. K. Chenchireddy, G. B. Mulla and S. A. Sydu, "Dual-Axis Solar Tracker Using Arduino and LDRs," Springer Nature Singapore, 2026. doi: 10.1007/978-981-95-0269-1_69